

智元G1 SDK & 使用手册 使用调查结果

机器人wifi地址：10.13.125.76

机器人debug网线地址：10.42.0.101

机器人 SLAM 地址：10.42.0.153:8080

机械臂示教器1：10.42.0.151:80

机械臂示教器2：10.42.0.152:80

摄像机调查（本调查并未结束，因为下述尝试并未达到效果）：

代码块

```
1  agi@localhost:~$ ls -la /dev/video*
2  crw-rw----+ 1 root video 81,  0 Jan  1  1970 /dev/video0
3  crw-rw----+ 1 root video 81,  1 Jan  1  1970 /dev/video1
4  crw-rw----+ 1 root video 81, 10 Jan  1  1970 /dev/video10
5  crw-rw----+ 1 root video 81, 11 Jan  1  1970 /dev/video11
6  crw-rw----+ 1 root video 81, 12 Apr 16 18:58 /dev/video12
7  crw-rw----+ 1 root video 81, 23 Apr 16 18:58 /dev/video13
8  crw-rw----+ 1 root video 81, 25 Apr 16 18:58 /dev/video14
9  crw-rw----+ 1 root video 81, 27 Apr 16 18:58 /dev/video15
10 crw-rw----+ 1 root video 81, 29 Apr 16 18:58 /dev/video16
11 crw-rw----+ 1 root video 81, 31 Apr 16 18:58 /dev/video17
12 crw-rw----+ 1 root video 81, 33 Apr 16 18:58 /dev/video18
13 crw-rw----+ 1 root video 81, 35 Apr 16 18:58 /dev/video19
14 crw-rw----+ 1 root video 81,  2 Jan  1  1970 /dev/video2
15 crw-rw----+ 1 root video 81, 37 Apr 16 18:58 /dev/video20
16 crw-rw----+ 1 root video 81,  3 Jan  1  1970 /dev/video3
17 crw-rw----+ 1 root video 81,  4 Jan  1  1970 /dev/video4
18 crw-rw----+ 1 root video 81,  5 Jan  1  1970 /dev/video5
19 crw-rw----+ 1 root video 81,  6 Jan  1  1970 /dev/video6
20 crw-rw----+ 1 root video 81,  7 Jan  1  1970 /dev/video7
21 crw-rw----+ 1 root video 81,  8 Jan  1  1970 /dev/video8
22 crw-rw----+ 1 root video 81,  9 Jan  1  1970 /dev/video9
```

启动服务：

代码块

```
1  agi@localhost:~$ systemctl list-unit-files --type=service | grep enabled
2  a2d_app.service                        disabled      enabled
3  a2d_mode_switch_server.service         enabled      enabled
4  a2d_nvme.service                       enabled      enabled
5
6  - a2d_app.service - A2D 应用服务
7  - genie_app.service - Genie 应用服务
8  - gui-service.service - GUI 服务
9  - agibot_perfguard.service - 性能守护服务
10 - rhino_start.service - Rhino 启动服务
11 - ota_master.service - OTA 升级服务
12
13
14 # 查看log
15 journalctl -u a2d_app.service --no-pager -n 50
16
```

机器人模式切换与相机配置分析

1. 模式切换机制概述

机器人的模式切换主要由以下几个组件协同工作：

组件	作用	位置
scene_control	模式切换核心工具	/home/agi/app/bin/scene_control
manifest.d	各模式的应用配置清单	/home/agi/app/conf/manifest.d/*.json
pbtxt配置	模式定义与相机模型选择	/home/agi/app/a2d_sdk/conf/*.pbtxt
deploy配置	相机详细参数配置	/home/agi/app/conf/deploy/*/

2. 模式配置文件详解

2.1 模式定义文件（pbtxt格式）

位置: `/home/agi/app/a2d_sdk/conf/`

copilot.pbtxt:

代码块

```
1 mode: COPILOT
2 hybrid_deploy_config {
3   use_cosine_sdk: true
4   camera_model: "copilot" # 指定使用 copilot 相机模型
5 }
```

vr.pbtxt:

代码块

```
1 mode: COPILOT
2 hybrid_deploy_config {
3   use_cosine_sdk: true
4   camera_model: "vr" # 指定使用 vr 相机模型
5 }
```

2.2 模式应用清单（JSON格式）

位置: `/home/agi/app/conf/manifest.d/`

每个模式定义了要启动的应用列表，以 **copilot.json** 为例：

代码块

```
1 {
2   "apps": [
3     {
4       "name": "Cosine",
5       "path": "/home/agi/app/bin/cosine_runner",
```

```

6         "arguments": ". camera_copilot", // 关键：指定相机配置
7         "env": "USE_PTP_CLOCK=OFF ...",
8         "delay": 10000
9     },
10    // ... 其他应用
11 ]
12 }

```

3. 相机配置详解

相机配置位于 `/home/agi/app/conf/deploy/` 目录下，按模式组织：

代码块

```

1  deploy/
2  |— camera_copilot/      # copilot 模式相机配置
3  |   |— rs_camera_conf_71.json    # Realsense 相机配置 (HW v7.1)
4  |   |— fisheye_camera_conf_71.json # 鱼眼相机配置 (HW v7.1)
5  |   |— rs_camera_conf_53.json    # Realsense 相机配置 (HW v5.3)
6  |   |— fisheye_camera_conf_53.json # 鱼眼相机配置 (HW v5.3)
7  |   |— scene.pbtxt           # 数据流配置
8  |— camera_preprocess/      # 预处理模式
9  |— develop/                # 开发模式

```

3.1 Realsense 相机配置 (`rs_camera_conf_71.json`)

代码块

```

1  {
2      "d455_1": { // 相机ID
3          "name": "head", // 相机名称 (头部相机)
4          "fps": "30", // 帧率
5          "width": "1280", // 宽度
6          "height": "720", // 高度

```

```

7      "depthEnable": true,           // 是否启用深度
8      "depthwidth": "640",          // 深度图宽度
9      "depthheight": "360",         // 深度图高度
10     "publish": true                // 是否发布数据流
11 }
12 }

```

3.2 鱼眼相机配置 (fisheye_camera_conf_71.json)

代码块

```

1  {
2      "cam1": {
3          "width": "1920",
4          "height": "1536",
5          "fps": "30",
6          "name": "hand_left_fisheye", // 左手鱼眼相机
7          "publish": true
8      },
9      "cam2": {
10         "name": "hand_right_fisheye", // 右手鱼眼相机
11         "publish": true
12     },
13     "cam4": {
14         "name": "head_center_fisheye", // 头部中心鱼眼相机
15         "publish": true
16     },
17     "cam5": {
18         "name": "head_left_fisheye", // 头部左侧鱼眼相机 (禁用)
19         "publish": false
20     },
21     "cam6": {
22         "name": "head_right_fisheye", // 头部右侧鱼眼相机 (禁用)
23         "publish": false
24     }
25 }

```

4. 修改相机配置的方法

4.1 修改现有模式的相机参数

步骤:

1. 确定要修改的模式 (copilot/vr/develop等)

2. 找到对应的配置文件:

- Realsense:

```
/home/agi/app/conf/deploy/camera_copilot/rs_camera_conf_*.json
```

- 鱼眼相机:

```
/home/agi/app/conf/deploy/camera_copilot/fisheye_camera_conf_*.js
```

3. 修改参数 (帧率、分辨率、是否发布等)

4. 重启服务或切换模式使配置生效

4.2 启用/禁用特定相机

修改 `publish` 字段:

- `"publish": true` - 启用该相机
- `"publish": false` - 禁用该相机

4.3 添加新相机

在对应的 JSON 配置文件中添加新的相机配置块:

代码块

```
1  {
2      "cam7": {
3          "width": "1920",
4          "height": "1080",
5          "fps": "60",
6          "name": "custom_camera",
7          "publish": true
8      }
```

```
9 }
```

4.4 切换相机模型

修改 `/home/agi/app/a2d_sdk/conf/*.pbt.txt` 文件中的 `camera_model` 字段：

代码块

```
1 camera_model: "copilot" // 改为 "vr" 或其他模型
```

5. 模式切换命令

代码块

```
1 # 切换到 copilot 模式
2 /home/agi/app/bin/scene_control switch copilot
3
4 # 切换到 vr 模式
5 /home/agi/app/bin/scene_control switch vr
6
7 # 切换到 idle 模式（仅启动基础服务）
8 /home/agi/app/bin/scene_control switch idle
9
10 # 重启当前模式
11 /home/agi/app/bin/scene_control restart copilot
```

6. 配置文件关系图



7.底盘地图扫描创建及说明

[📖 智元G1底盘使用手册](#)

8.机械臂示教器地址

[📖 智元机器人手臂示教器](#)

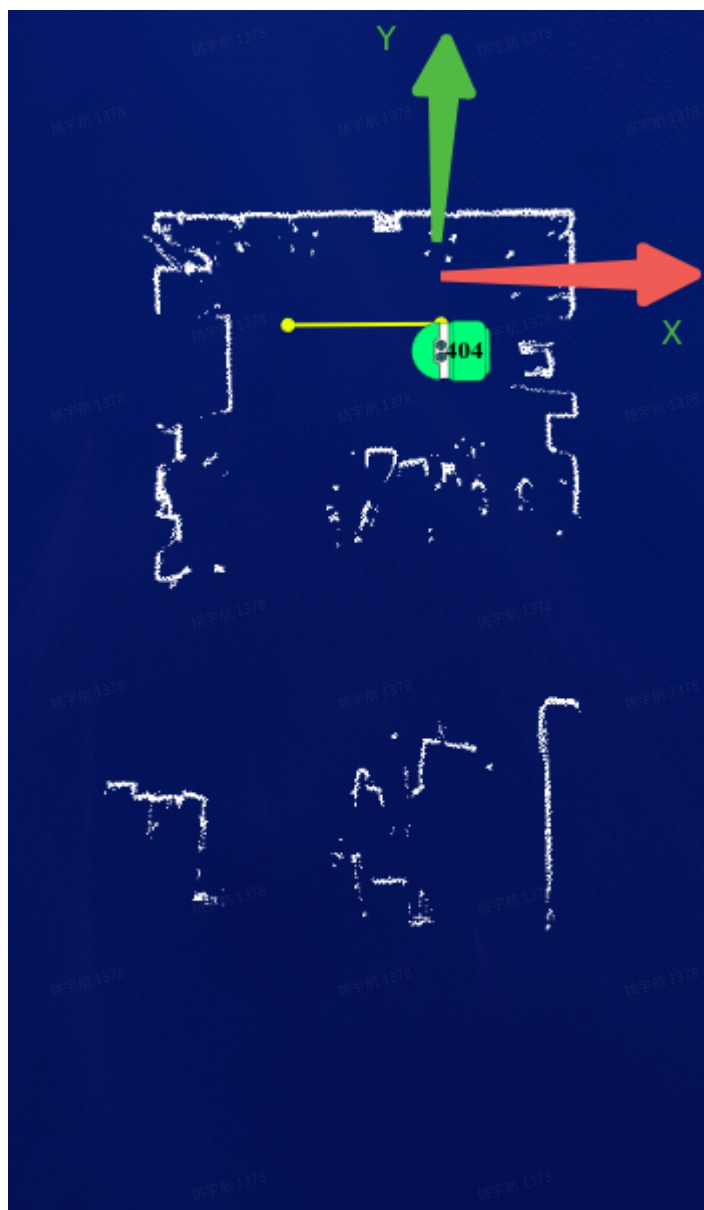
总结

需求	操作位置
修改相机分辨率/帧率	conf/deploy/camera_copilot/*_camera_conf_*.json
启用/禁用相机	修改 <code>publish</code> 字段
切换相机模型	a2d_sdk/conf/*_pbtxt 中的 <code>camera_model</code>
修改启动应用列表	conf/manifest.d/*_json
执行模式切换	scene_control switch <mode>

SLAM 模块

坐标系：

- i. 右手坐标系。坐标系的建立和地图扫描时机器人的初始位置相关。



几个keypoint:

代码块

- 1 (768,199)
- 2

代码块

- 1 # 导入slam包

```

2  from a2d_sdk.robot import Slam
3  slam = Slam()
4  # 注意切换到自动导航模式前，一定要确保机器人周边安全，以及 SLAM 管理页面中 任务信息列表
   中没有任务
5  slam.switch_nav_mode(2)
6
7  # 绝对移动位置 起始点
8  slam.navigate_to_pose(587,270,0)
9  # 绝对移动位置 终止点
10 slam.navigate_to_pose(-1308,0,180)

```



代码块

```

1  slam.get_chassis_position()
2  ({'agv_status': 8, 'position_conf': 92.0, 'agv_pos_x': 0.6620000004768372,
   'agv_pos_y': 0.03999999910593033, 'agv_pos_z': 0.0010000000474974513,
   'agv_angle': 3.1326241493225098, 'odom_x': -0.014000000432133675, 'odom_y':
   -0.0729999989271164, 'odom_z': 0.0, 'odom_angle': 0.01081900019198656,
   'linear_speed': 0.0, 'angular_speed': 0.0, 'acc_x': -0.31299999356269836,
   'acc_y': 0.20499999821186066, 'acc_z': 9.83899974822998, 'gyro_x': 0.0,
   'gyro_y': 0.0, 'gyro_z': 0.0, 'roll': 1.8509999513626099, 'pitch':
   1.4880000352859497, 'yaw': 0.6899999976158142, 'motor_states': [{'id': 4,
   'enable': False, 'position': 0.0, 'velocity': 0.0, 'effort': 0.0,
   'current': 7.174648137343064e-43, 'voltage': 0.0, 'temperature': 30.0,
   'status': 0, 'err_code': 0}, {'id': 5, 'enable': False, 'position': 0.0,
   'velocity': 0.0, 'effort': 0.0, 'current': 7.174648137343064e-43,

```

```

'voltage': 0.0, 'temperature': 30.0, 'status': 0, 'err_code': 0}},
'agv_task_state': {'task_uuid': 0, 'task_reqid': '', 'curr_station_idx': 0,
'finish_state': 0}}, 1.7766515508096617e+18)

3
4
5
6 >>> slam.get_chassis_position()
7 ({'agv_status': 5, 'position_conf': 99.0,
8  'agv_pos_x': -1.2710000276565552, 'agv_pos_y': 0.009999999776482582,
9  'agv_pos_z': 0.00100000000474974513, 'agv_angle': 3.2013771533966064,
10 'odom_x': 8.092000007629395, 'odom_y': -4.041999816894531,
11 'odom_z': 0.0, 'odom_angle': -0.6552475094795227,
12 'linear_speed': 0.0, 'angular_speed': 0.0,
13 'acc_x': -0.3619999885559082, 'acc_y': 0.5189999938011169,
14 'acc_z': 9.83899974822998, 'gyro_x': 0.0, 'gyro_y': 0.0,
15 'gyro_z': 0.0, 'roll': 2.055000066757202, 'pitch': 2.7190001010894775,
16 'yaw': -37.48400115966797,
17 'motor_states': [{'id': 4, 'enable': False, 'position': 0.0, 'velocity':
0.0, 'effort': 0.0, 'current': nan, 'voltage': 0.0, 'temperature': 30.0,
'status': 0, 'err_code': 0},
18 {'id': 5, 'enable': False, 'position': 0.0, 'velocity': 0.0, 'effort': 0.0,
'current': nan, 'voltage': 0.0, 'temperature': 30.0, 'status': 0,
'err_code': 0}],
19 'agv_task_state':
20 {'task_uuid': 0, 'task_reqid': '', 'curr_station_idx': 0, 'finish_state':
0}}, 1.7766614809787443e+18)

21
22
23 dir(slam)
24 ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattr__', '__gt__', '__hash__',
'__init__', '__init_subclass__', '__le__', '__lt__', '__module__',
'__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
'__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__',
'slam', 'delete_map', 'get_arm_col_level', 'get_chassis_battery_status',
'get_chassis_info', 'get_chassis_position', 'get_latest_map', 'get_map',
'get_map_list', 'mapping_control', 'move_to_relative', 'navigate_to_pose',
'pause_walk', 'relocation', 'set_agv_col_ctrl', 'set_agv_col_level',
'set_arm_col_level', 'set_chassis_info', 'switch_map', 'switch_nav_mode']

```

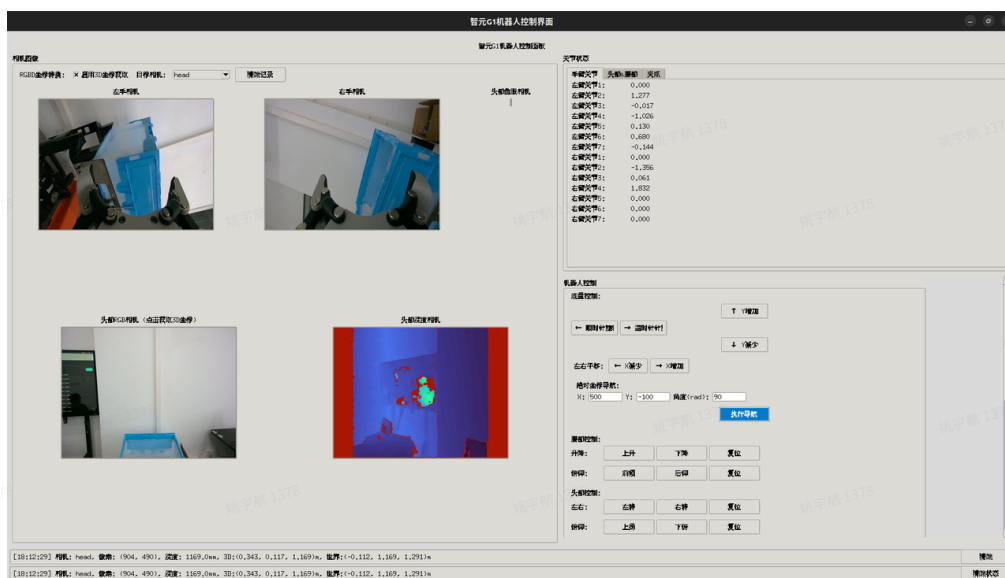
机器人控制程序使用说明:

a. 使用环境 Ubuntu 22.04

b. 安装GDK

代码块

```
1 cd /opt/workspace/G1_SDK_ENV
2 source /opt/ros/humble/setup.bat
3 conda activate GDK
4 source ./a2d_sdk/env.sh
5 python robot_control_gui.py
```



1. 机械臂各关节状态区：

关节状态	
手臂关节	头部&腰部 夹爪
左臂关节1:	0.000
左臂关节2:	1.277
左臂关节3:	-0.017
左臂关节4:	-1.026
左臂关节5:	0.130
左臂关节6:	0.680
左臂关节7:	-0.144
右臂关节1:	0.000
右臂关节2:	-1.356
右臂关节3:	0.061
右臂关节4:	1.832
右臂关节5:	0.000
右臂关节6:	0.000
右臂关节7:	0.000

2. 执行状态区：

[17:44:02] 相机: head, 像素: (1036, 523), 深度: 625.0mm, 3D:(0.275, 0.085, 0.625)m, 世界:(-0.046, 0.625, 1.254)m	清除
[17:44:02] 相机: head, 像素: (1036, 523), 深度: 625.0mm, 3D:(0.275, 0.085, 0.625)m, 世界:(-0.046, 0.625, 1.254)m	清除状态

3. 底盘控制区：

机器人控制

底盘控制:

← 顺时针!

→ 逆时针!

↑ Y增加

↓ Y减少

左右平移:

← X减少

→ X增加

绝对坐标导航:
X: Y: 角度(rad):

执行导航

4. 腰部 头部 夹爪控制区：

腰部控制:

升降:

上升

下降

复位

俯仰:

前倾

后仰

复位

头部控制:

左右:

左转

右转

复位

俯仰:

上扬

下俯

复位

夹爪控制:

左夹爪:

张开

闭合

右夹爪:

张开

闭合

5. 机械臂控制部分：

注意：

- i. 该部分控制只有在develop模式下可以使用
- ii. 安全模型（RRT规划）模式FR在开发中，写的不好目前不可用。
- iii. 预设: 预设的3个动作运动幅度都特别大，注释掉了。

- iv. 获取当前位姿（厂商SDK不支持，开发时还没有URDF，用的简化算法。未来根据URDF计算）

机械臂末端控制：

左臂：

右臂：

预设：

末端位姿控制

选择手臂： ☒ 左臂 ☐ 右臂

位置（米）： X: Y: Z:

姿态（弧度）： Roll: Pitch: Yaw: ☐ 安全模式（RRT规划）

6. RGBD 数据区：（开发中）

RGBD坐标转换结果

点击图像获取3D坐标...

7. RGBD 相机选择区：

相机图像

RGBD坐标转换： ☒ 启用3D坐标获取 目标相机：

选择“启用3D坐标获取”后，在RGB区点击某点可以获得该点在真实世界坐标（相对机器人底盘）。

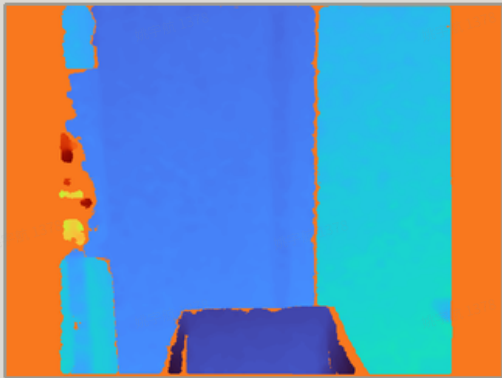
注意：

- 深度相机VOF小于RGB相机，所以画面左右两条没有深度信息。
- 用获得的真实世界空间坐标 (x,y,z)后，用来控制End Effector Pose时注意需要坐标变换。

头部RGB相机 (点击获取3D坐标)



头部深度相机



[18:39:33] 相机: head, 像素: (736, 683), 深度: 925.0mm, 3D:(0.099, 0.291, 0.925)m, 世界:(0.146, 0.925, 1.445)m

[18:39:33] 相机: head, 像素: (736, 683), 深度: 925.0mm, 3D:(0.099, 0.291, 0.925)m, 世界:(0.146, 0.925, 1.445)m